An Analysis Of Resource-aware Parallelization Techniques On Heterogeneous Environments

Hyunjeong Kim
Dept. of Statistics
Sookmyung Women's University
Seoul, Korea
hjkimmm@sookmyung.ac.kr

Jiwon Ha
Dept. of Computer Engineering
Seoul National University
Seoul, Korea
jwh0245@naver.com

Theodora Adufu
Dept. of Computer Science
Sookmyung Women's University
Seoul, Korea
theoadufu@sookmyung.ac.kr

Yoonhee Kim
Dept. of Computer Science
Sookmyung Women's University
Seoul, Korea
yulan@sookmyung.ac.kr

***Abstract*— With the proliferation of Deep Learning (DL) models across diverse fields, there arises a need to study model optimization methods to enhance accurate inferences. In this study, resource-aware parallel optimization techniques are applied to DL models to reduce the time required for model training and inference as well as maximizing the utilization of heterogeneous computing resources. Through profiling of DL models using DCGM and analysis of existing parallel optimization techniques like DDP, we conducted a study to maximize the use of heterogeneous computing resources.**

***Keywords*—Deep Learning, heterogeneous, DDP, GPU, NLP**

I. INTRODUCTION

Sequel to the success in developing Deep Learning (DL) models, research using DL models in various fields such as natural language processing and computer vision have become common practice leading to the development of larger models. As the sizes of models increase, resources such as multi-GPUs are used for model processing. Nevertheless, larger GPT-3 models, trained on extensive datasets, necessitate fine-tuning for accurate inference in cumulative learning paradigms due to the presence of unverified information within the training data. [1] Consequently, there arises a pressing need to optimize these models to swiftly adapt to additional training data, particularly in heterogeneous computing environments, where training time and model learning curve length can vary depending on the computational capacity of the computing device.

In heterogeneous environments, data must be properly segmented to maximize parallelism for resources of different computing functions. Using data parallelization techniques alone without considering the hardware environment creates bottlenecks as the model size increases.

In this study, we study how data size affects the model's training time in heterogeneous environments. First, the optimized BERT model is profiled using the DCGM profiling tool, and GPU resource usage such as SM and Memory is observed [2].

We then study resource-aware parallel optimization techniques that can maximize resource utilization in heterogeneous resource environments and shorten the training time of deep learning models through distributed data parallelism (DDP).

II. RELATED RESEARCH

A. Prior Experiment

We observed the resource utilization when NLP models such as BERT is deployed on different resources in the process of training and inference. We investigated the operation of BERT[2] in 1g.6gb (1 Multi-Instance GPU, MIG), 2g.12gb (2MIG), and multi-GPU (2GPU) environments. In all experiments, the verification results maintain an error level of 2.0e-06.

We observed that the average utilization of GPU resources such as SM and memory is relatively low when BERT runs on different GPU resources, as shown in Figure 1. In addition, we found that while there are noticeable differences in SM activity (SMACT) across GPU resources, DRAM activity (DRAMA), which shows data movement over memory bandwidth, is almost identical across resources. Next, we will study the effect of data size on the performance time of the model in a heterogeneous environment.

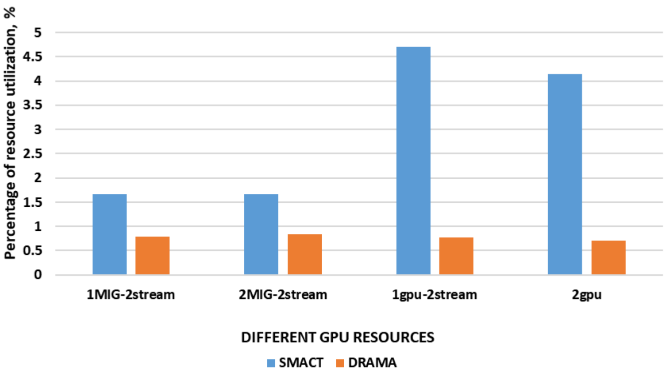

Fig. 1. Average SM and Memory resource utilization of BERT models according to GPU resource environment

B. *Proposed approach*

In this our focus is on investigating how Distributed Data Parallel (DDP) techniques influence performance within heterogeneous resource environments. Our proposed approach follows the basic model learning structure as shown in Figure 2, but considers implementation in heterogeneous resources.

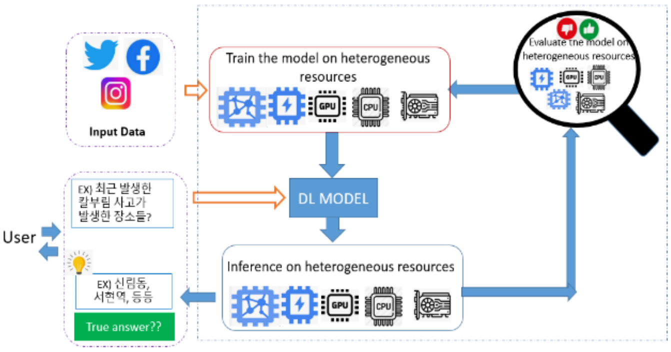

Fig. 2. Proposed Model

DDP is a data parallelism technology in which datasets for model learning are shared between multiple GPUs on a single node or between heterogeneous GPUs in a multi-node environment. While using DDP in heterogeneous environments, data is pinned to CPU memory to maximize asynchronous movement of data for different types of GPUs.

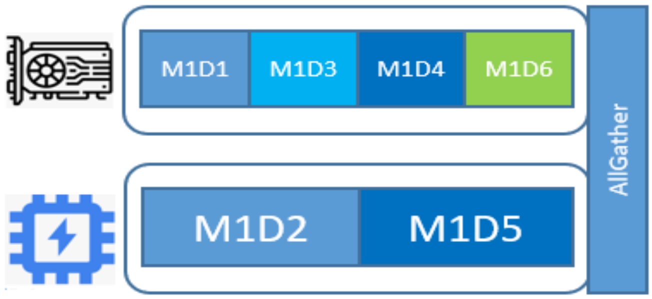

Fig. 3. Data Parallelism in heterogeneous environments

Subsequently, the dataset is divided into all available GPU resources through the distributed data sampler. The distributed data sampler ensures that data does not overlap between GPU resources. As shown in Figure 3, the same model (M1) is distributed to all GPU resources to perform each task on different datasets (D1-D5), and the training results are returned through the All-Gather process. Data movement between GPUs takes place through NV-Link. DDP is particularly efficient when training large-sized models with vast amounts of data in the Multi-GPU model. Previous studies such as Whale[4] identify dependencies between models and propose a method of combining data parallelization techniques and model parallelization techniques as shown in Figure 3 (during executions) in heterogeneous environments.

C. *Experiment Result*

First, training is conducted for the minGPT[7] NLP model on NODE A and NODE B, respectively, through PyTorch's DDP technique. NODE A includes NVIDIA A30(Compute capability 8.0, Device memory 24GB) 2 GPU units and NODE B has NVIDIA TITAN XP(Compute capability 8.0, Device memory 24GB) 4 GPU units. For the dataset, we use TinyShakespeare, the benchmark dataset of CharGPT.

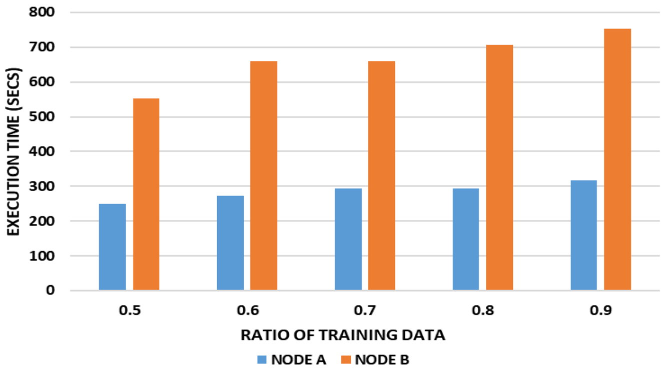

Fig. 4. Comparison of run times by execution environment

Leveraging the Torchrun package, the minGPT model is executed to divide the training and evaluation dataset into various percentages for each node. Figure 4 shows the total execution time measurement results for NODE A and B, and Figure 5 shows the learning loss and evaluation loss in each case.

Experiments confirmed that despite having more GPUs in NODE B, the performance time of minGPT in NODE B was twice the performance time in NODE A. This observation underscores the significance of considering GPU performance when distributing data across distinct GPU types.

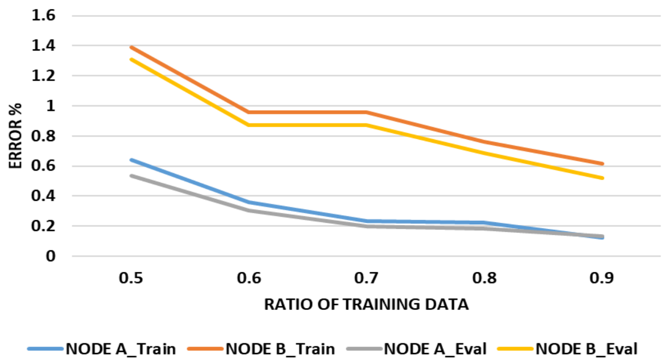

Fig. 5. Comparison of learning loss and evaluation loss according to execution environment

Furthermore, in the case of NODE B, it was observed that the error rate increased in both training and evaluation stages as the proportion of training data decreased.

III. Conclusion

According to the experimental results of this study, the application of DDP techniques to model learning can only be effective when the performance of the underlying resource is high. The size of the dataset should also be considered to improve accuracy while reducing training time. Future research will be conducted on additional research on resource usage of DL models other than NLP models and other parallel optimization technologies such as Fully Sharded Data Parallel (FSDP).

This paper was studied with the support of the Ministry of Science and ICT and the Korea Women's Science and Technology Development Foundation with government funding in 2023 (a project to support the engineering research team system for female college students at the Ministry of Science and ICT).

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2021R1A2C1003379).

References

[1] "Microsoft launches the new Bing, with ChatGPT built in", https://techcrunch.com/2023/02/07/microsoft-launches-the-new-bing-with-chatgpt-built-in/ (Last accessed 6th March, 2023)

[2] NLP-Fast, https://github.com/SNU-HPCS/NLP-Fast/tree/main

[3] Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. 2019. MnnFast: a fast and scalable system architecture for memory-augmented neural networks. In Proceedings of the 46th International Symposium on Computer Architecture (ISCA '19). Association for Computing Machinery, New York, NY, USA, 250–263. https://doi.org/10.1145/3307650.3322214

[4] Wang, Ang & Jia, Xianyan & Jiang, Le & Zhang, Jie & Li, Yong & Lin, Wei. (2020). Whale: A Unified Distributed Training Fram9ework.

[5] “Getting Started with Distributed Data Parallel — PyTorch Tutorials 2.0.1+cu117 documentation”, https://pytorch.org/tutorials/intermediate/ddp_tutorial.html

[6] https://pytorch.org/tutorials/intermediate/ddp_tutorial.html

[7] minGPT, https://github.com/karpathy/minGPT